DATA AND INFORMATION QUALITY PROJECT REPORT

PROJECT ID: 61
ASSIGNED DATASET: 3

Zhongyou Liang 10905937
Alessandro Turazza 10768675

Contents

1 Setup Choices

1.1 Libraries and Tools

- **Python Libraries:**
  - `Pandas`: Data manipulation and cleaning.
  - `ydata_profiling`: Data profiling and exploratory analysis.
  - `Spacy`: Text processing and entity extraction.
  - `Recordlinkage`: Data deduplication and record matching.
  - `Sklearn`: Machine learning for predictive imputation of missing values.
  - `Re`: Library for removing special characters and find complex patterns
  - `Numpy`: Library used for standard null value (nan)
- **Environment**: Google Colab for execution and Google Drive for data storage.

2 Pipeline Implementation

2.1 Data Exploration

The dataset is in the shape of 6094 rows, 22 columns. We used ydata-profiling to do profiling because it is convenient and it provides comprehensive dataset overview. We calculated uniqueness, distinctness, constancy and completeness for every column to do data quality assessment. The following issues were observed in the dataset after a comprehensive data profiling and data quality assessment.

1. **Low Uniqueness**
   Columns like Year, Type, Country, and Sex have very low uniqueness values, indicating a high degree of repetition in their values. This suggests limited variability or categorical data with few distinct options.

2. **Low Distinctness**
   Columns like Age, Type, and Sex show low distinctness, implying some values are disproportionately repeated compared to the total number of non-null entries. This could hint at skewed distributions.

3. **High Constancy**
   Columns like Sex, Type, and Fatal (Y/N) have high constancy values, indicating that a single value dominates the column. This might make these columns less useful for analysis, as they lack sufficient variability.

4. **Completeness Issues**
   Some columns' completeness are not 1, indicating missing value in those columns.

5. **Potential Redundancy**
   Some fields, such as Case Number.1, Case Number.2, href formula, and original order, show near-perfect uniqueness but might be redundant or unnecessary duplicate. Case Number.1 and Case Number.2 may contain identical information, href

formula and href may overlap in content, original order is uniformly distributed, only serve as an index and not contribute to the dataset's primary analysis. These fields may unnecessarily inflate the dataset without adding value.

2.2 Data Cleaning

2.2.1 Data Transformation

We saw that there are some leading/trailing spaces and specific symbols in the data and some empty string in the dataset. Thus, before dealing with specific columns, we removed leading/trailing spaces, replacing empty strings with nan value and removing specific symbols (@#$%*!?) for every column.

1. **Case Number column**
   Case Number, Case Number.1 and Case Number.2 may contain identical information. We compared them and found they are not equal but highly similar, and we saw Case Number just an identifier for the case and no need to do further transformation. Thus, we dropped Case Number.1 and Case Number.2, just kept Case Number column.

2. **Date Column**
   The Date column has low constancy and the format is really inconsistent. The format in this column mainly are:

   - "YYYY-MM-DD", e.g. 1900-01-01
   - "YYYY.MM.DD", e.g. 1900.01.01
   - "Season and year", e.g., "Winter 1942"
   - "mid-year", e.g., "mid-1870s"
   - "day-month-year", e.g., "Reported 03-Sept-1816"
   - "month-year", e.g. "xxxSept-2013xxx"
   - "year", e.g., "xxx1900xxx"

   They need to be standardized. We want them all to be converted to the format "YYYY-MM-DD". We wrote a function to standardize them, using regular expression to match the text. If data already good, return directly. If data in "YYYY.MM.DD" format, convert them. If data in "Season and year" format, retrieve season and year, do season mapping, seasons_mapping = {"Winter": "12-01","Spring": "03-01","Summer": "06-01","Fall": "09-01"}, then convert. If data in "mid-year" format, set mid as "06-01", then convert.If day, month,and year appear in the data, retrieve them and convert to "YYYY-MM-DD". If just month and year appear in the data, retrieve month and year, set day as "01", then covert. If just year appear, retrieve year, set day and month both as "01", then covert. Sometimes, the spelling of month is not correct in the data, before we matched the text, we corrected them. After aaplying the standardized function on the Date column, it become consistent, then we parsed it to the "datetime" datatype.

3. **Year column**
   We found that there are some rows that the year value is not the same in Date

column and Year column. And there are some zeros in Year column. We thought that data in the Date column is more correct. In this step, we just converted data in Year column as integer. In the following step, we replaced the data in Year column with the year value in Date column. We didn't do this in this step because we also found there are outliers in the Date column. So we did in the following step.

4. **Time column**
   We thought Time column is related to Date column, so we used pop and insert to move it next to the Date column. Time column is also really messy. We used regular expression to match the text and cover them to the consistent format in "HH-MM". We used timemap={ 'early morning': '06:00', 'late morning': '10:30', 'morning': '08:00', 'early afternoon': '13:00', 'late afternoon': '16:30', 'afternoon': '15:00', 'late evening': '21:00', 'evening': '19:00', 'dusk': '18:30', 'night': '22:00', 'noon': '12:00', 'midnight': '00:00' } to do time-mapping. For other cases, we retrieved minute and hour in the data and then covert.

5. **Country column**
   This column was already quite clean, we only decided to take all the strings to a title format and replacing some values like "England", "Scotland", "Ceylon" to their political nation ("United Kingdom" and "Sri Lanka"). After that we solve the problem of the format country1/country2 taking only the first country in order to have a unique format made only by a string containing one nation.

6. **Area column**
   This column contains many different formats, we decided to remove each illegible or special characters (like '?' or '(' maintaining only the letters, spaces, hyphens, apostrophes), numbers and special strings like "Km", "Ft", "Miles" and "Yard", this because we assume to have a unique format, considering the approximation of these value as the real value: i.e. "16 miles off Hilton Head" -¿ "Hilton Head". After that we applyed a strip and then we took all the rows to capital letters.

7. **Location column**
   This column in a very similar way to Area contains many different formats, so like before we decided to take only the letters, strip and take the rows to title format. In this case some rows contains only the precise location, while others contains only the county and still others contains both location and county, since we don't have a domain in order to automatically recognize them we decided to take all of them in the cleaned dataset.

8. **href and href formula column**
   href formula and href may overlap in content. We compared this two column and found that they are highly similar. When we saw the difference between this two columns, we found that many data in the href column having "http" two times in one row, like "http://sharkattackfile.net/spreadsheets/pdf_directory/http://sharkattackfile.net/spreadsheets/pdf_directory/2017.06.11-Goff.pdf". So, data in href formula column is more correct. When data in href formula column is not correct, we saw that data in the href column is correct, so we replaced them. Then we found the rows that in href formula column is not start with "http", we corrected them. Finally, we dropped href column, kept href formula column and rename it as href column, because "href" is more meaningful.

9. **Original order column**
   Delete the column 'original order' as it was uniformly distributed, not useful.

10. **Type and Activity column**
    Type column is related to Activity column, so we moved it next to the Activity column and deal with them together. We saw that there many categories of activity in Activity column, and the data having so many description of the activity. We created a new column called Activity Detail having same value as Activity does, then we divided the data in Activity column as the categories "Swimming, Diving, Surfing, Boarding, Fishing, Wading, Walking, Standing, Snorkeling, Bathing and Other". We set data in the Activity Detail column as "No detail" when data is same in Activity and Activity Detail column. Thus, data in Activity column will categories, and data in the Activity Detail column will be description of activity. We see that in Type column, the data are 'Unprovoked', 'Provoked', nan, 'Invalid', 'Boat', 'Sea Disaster' and 'Boating'. We compared Type, Activity and Activity Detail column based on the class in Type column, and found that when data in Type column is Boat, Boating and Sea Disaster, data in Activity column sometimes are Other. In this case, we set boat as boating in Type column, and set Other in Activity column as the same value in the Type column. Then when data in Type column is Boating or Sea Disaster, set them as Unknown. Thus, data in Type column will be 'Unprovoked', 'Provoked', nan, 'Unknown'.

11. **Name column**
    In this column we decided to remove, as usual, each illegible character and then we identify the rows containing values like "male","female"... in order to use them in the Sex row, after that we subsitute them with nan value. Then we use Spacy library in order to distinguish personal name with common names (i.e. "A scotland"), in particular we took each personal name and replace the others with the nan value.

12. **Sex column**
    First of all we rename this column removing the space, after that we notice that there are only few values: ['M', 'F', nan, 'lli', 'N', '.'], we decided to take only the ones that we are sure: 'M' or 'F', while the others have been replacing with nan.

13. **Age column**
    in this column we decided to take only one number, to do so we strip all the rows for some final 's' (i.e. "20s" becomes "20"), and convert "Teen" to a standard value of "18". For all the others format we decided to take only the first number we encounter discarding the rest of the string, where it results an empty string we took it to nan.

14. **Injury and Fatal(Y/N) column**
    Injury and Fatal column are related, we deal with them together. We removed 'PROVOKED INCIDENT' and the associated comma in the data of Injury column, because provoked or not already appear in the Type column. Data in the injury column is a description of the injury, they can be inconsistent because every case is not the same, so we did not do further transformation. For Fatal(Y/N) column, data it it are 'N', 'Y', nan, 'UNKNOWN', '2017', 'F', 'VALUE', 'n'. We renamed Fatal(Y/N) as Fatal, because it is more concise, and set Y as YES, N as NO. We checked '2017', 'F', 'VALUE', 'n' in Fatal column, set them to YES or NO according

to the data in Injury column is fatal or not. Then data in Fatal column become 'NO', 'YES', nan, 'UNKNOWN'.

15. **Species column**
    First of all we rename this column removing the space, after that we noticed that many rows contains the size of the shark beyond their species, thus we decided to extract the size, placing it in a new column "Size [m]". In the case where there is a multiple size we took only the first (i.e. 8-12 becomes 8) and then, since there are both meters and feet format, we decide to take only the meters format casting the feet into meters, if a number is not present we took it to nan. After that we decide to take a unique format of the species relying on a little self-made dataset containing the most important shark species called "SPECIES", in particular, for each row, we check if there is a match between the row and a particular specie in the dataset, if so we raplace the whole string with only the specie, else we replace the string with nan.

16. **Investigator or Source column**
    For this column we decided to take only either the names of the authors or the journalistic house, removing the dates and the pages format (i.e. Pp.123-124), then we took each row to title format, removing some special characters like '?' and the empty value comma (", ,").

17. **pdf column**
    For this column, since we have no way to check if a source exists or not, the only thing we have done is to check if the rows end with ".pdf". Luckily we found that only few rows hasn't this format so we were able to recognize all of them, thus for each of these very few different formats we replace the final part of the string with ".pdf".

2.2.2 Error detection & Correction missing values

There are missing values in Date, Time, Country, Area, Location, Type, Activity Detail, Name, Sex, Age, Injury, Fatal, Species, Size[m], Investigator or Source column.

We used "ffill" to fill missing values in Date column. "ffill" Fills missing values with the most recent non-missing value. We use it because in Date column, adjacent data are usually close in time. We filled missing values in Time column by "00:00:00", because we didn't know the exact value and this make data consistent in Time column. We used the most frequent data which is "Unprovoked" appearing in Type column to fill. For Size[m] column, we used the mean value to fill. For Activity Detail column, We filled "No detail" in missing values. For Sex and Age column, we filled "Not Specified", and the remaining column, we filled "Unknown". For there kinds of columns, it is difficult to infer their missing values, so, we filled in this way.

We try to use machine learning techniques(Random Forest) to fill missing value in Age column and Species column, but the results are not good enough. We tested the mean squared error of the model to fill Age column, which is 229.08, it is terrible. And we tested the validation accuracy of the model to fill Species column, which is 0.5886, it is not so good. So we still used the previous way to fill missing value on these columns.

2.2.3 Error detection & Correction of outliers

We found outliers in Date column. There are some date greater than today, e.g., "2028-04-06", indicating that they are outliers. We found the max year of Year column, and found date greater than the max year in Date column. We saw that when date greater than max year, year value in the Case Number column and Year column are the same, which means these year values are more correct, so we used year value in Year column to replace the year in Date column when it is outlier. Then, because there are some zeros in Year column, we use year value in Date column to replace Year column.

2.2.4 Data Deduplication

We used recordlinkage Python library to deal with data duplication because it is an excellent tool for identifying and resolving duplicate entries in a dataset, especially when exact matches are not possible due to slight variations in the data.

We sorted on 'href' column, because records with very different hrefs are unlikely to be duplicates, and hrefs are one of the most distinct attributes for matching records. We set the window size as 21, because after this number, the matching records not increased. We set Set the threshold for matches as 15, because we regarded similarity greater than 80% as duplication. Then we dropped the duplications.

3 Results

1. **Completeness**
    - Before Cleaning: Some columns had completeness below 1.0, indicating missing or null values.
    - After Cleaning: All columns now have a completeness of 1.0, showing that missing data has been effectively addressed.
    - Impact: This ensures that every row in the dataset has a value for every column, improving the reliability of any downstream analysis or modeling.
2. **Redundancy Reduction**
    - Before Cleaning: Redundant columns like Case Number.1, Case Number.2, and possibly others appeared in the dataset.
    - After Cleaning: These have been removed or consolidated into a single column (e.g., Case Number), simplifying the dataset.
    - Impact: Reduces confusion, decreases storage requirements, and improves processing efficiency.
3. **Better Handling of Dominant Categories**
    - Before Cleaning: Columns like Type, Sex, and Fatal had extremely high constancy, suggesting that a single value dominated the dataset.
    - After Cleaning: While the dominance remains (e.g., Type has a constancy of 0.733), these columns appear cleaner, with outliers or inconsistencies removed.

- Impact: Ensures that the dominant values accurately represent the dataset's characteristics.

4. **Uniform Data Types and Formats**
    - Before Cleaning: Possible issues with mixed data types or formats (e.g., string, numeric inconsistencies).
    - After Cleaning: Columns now have been standardized, facilitating smoother operations like statistical analysis and visualization.
    - Impact: Avoids errors during processing and ensures compatibility with analytical tools.